

■ Prazo máximo **7 dias**
corridos

■ Entregas antecipadas
serão avaliadas primeiro

■ Stack obrigatória
NestJS · Prisma ·
PostgreSQL

■ Entrega **Repositório Git**
público

1. Contexto do Desafio

A Commandix está construindo uma plataforma SaaS multi-tenant para gestão de contratos. Neste desafio, você irá desenvolver o módulo central dessa plataforma: um sistema onde cada tenant (empresa cliente) possui seus próprios contratos, campos configuráveis e histórico de alterações, tudo isolado e acessível apenas pelos usuários daquele tenant.

2. Funcionalidades Requeridas

2.1 Autenticação & Multi-tenancy

- Cadastro e login de usuários com JWT (access token + refresh token).
- Cada usuário pertence a um tenant. Um usuário só enxerga dados do próprio tenant.
- Suporte a pelo menos dois papéis: **Admin** (gerencia tudo no tenant) e **Viewer** (apenas leitura).
- Rota de criação de tenant + primeiro usuário Admin (onboarding).

2.2 Contrato Padrão (Template)

- Cada tenant possui um **contrato padrão** — um template com campos configuráveis.
- O Admin pode definir quais campos compõem o contrato: nome do campo, tipo (texto, número, data, booleano) e se é obrigatório.
- O template é editável a qualquer momento. Alterações no template **não alteram retroativamente** contratos já gerados.

2.3 Geração e Preenchimento de Contratos

- A partir do template ativo, o usuário pode criar uma instância de contrato preenchendo os campos definidos.
- Validação dos campos obrigatórios e dos tipos antes de salvar.
- Um contrato gerado possui status: **Rascunho**, **Ativo** ou **Encerrado**.

2.4 Histórico de Alterações

- Toda alteração em um contrato (criação, edição de campo, mudança de status) deve ser registrada.

- O histórico deve conter: o que mudou, qual campo, valor anterior, valor novo, quem alterou e quando.
- Endpoint para listar o histórico de um contrato específico.

2.5 Listagem e Filtros

- Listagem de contratos do tenant com paginação.
- Filtro por status e por intervalo de datas de criação.
- Busca por valor de campo (ex: buscar contratos pelo nome do cliente).

3. Requisitos Técnicos

Backend

- **NestJS** com TypeScript — obrigatório.
- **Prisma ORM** com **PostgreSQL** — obrigatório.
- Arquitetura em módulos: AuthModule, TenantModule, ContractModule, HistoryModule.
- Validação de DTOs com **class-validator** e **ValidationPipe** global.
- Guards para autenticação JWT e controle de roles (RBAC).
- Migrations versionadas via Prisma — o banco deve subir do zero com **prisma migrate deploy**.

Frontend

- **React** com TypeScript — obrigatório.
- **Vite** como bundler — obrigatório.
- Telas mínimas: Login, Dashboard do tenant, Configuração do template, Lista de contratos, Detalhe + histórico do contrato.
- Consumo da API REST produzida no backend.
- Qualidade visual será avaliada, mas não é o foco principal.

Infraestrutura

- **Docker Compose** com todos os serviços: API, frontend e banco — **um único comando deve subir tudo**.
- Arquivo **.env.example** com todas as variáveis necessárias documentadas.
- README claro com instruções de setup, seed e acesso.

4. Critérios de Avaliação

Critério	Peso	O que observamos
Qualidade do código	Alto	Organização, nomenclatura, separação de responsabilidades
Arquitetura NestJS	Alto	Módulos, Guards, Pipes, injeção de dependência
Modelagem do banco	Alto	Schema Prisma, relações, índices, migrations

Multi-tenancy	Alto	Isolamento real de dados entre tenants
Histórico de alterações	Médio	Rastreabilidade, estrutura do log
Docker Compose	Médio	Sobe com um comando, healthcheck, variáveis
Frontend funcional	Médio	Fluxo completo usável, sem travar
Testes	Bônus	Unitários ou E2E — não obrigatório, mas valorizado
README	Bônus	Clareza, decisões arquiteturais documentadas

5. Forma de Entrega

- Repositório **Git público** (GitHub, GitLab ou Bitbucket).
- Envie o link do repositório por e-mail ou WhatsApp para o recrutador responsável.
- O repositório deve conter o código do backend, frontend e o **docker-compose.yml** na raiz.
- Inclua um arquivo **README.md** com: como rodar, credenciais de acesso ao seed, e decisões técnicas relevantes.
- Inclua dados de seed: ao menos **2 tenants**, **3 usuários** e **5 contratos** de exemplo.

6. Dicas e Observações

■ Entrega antecipada Candidatos que entregarem antes do prazo terão suas POCs avaliadas prioritariamente. Não espere os 7 dias se estiver pronto.	■ Foco no essencial Prefira funcionalidades completas e bem feitas a muitas funcionalidades pela metade. Um módulo sólido vale mais que cinco módulos quebrados.
■ Documente decisões Se optou por uma abordagem específica de multi-tenancy ou histórico, explique no README o porquê. Demonstra maturidade técnica.	■ Docker é obrigatório Projetos que não sobem com docker compose up não serão avaliados. Teste em uma máquina limpa antes de enviar.